DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Link, Start!

CMSC 197 GDD Activity 1

Prepared by: Rene Andre Bedonia Jocsing

Published: January 27, 2026

Welcome to **CMSC 197: Game Design and Development**. This course uses the **Godot Engine** and its integrated scripting language, **GDScript**. Your first activity introduces you to GDScript fundamentals through GDQuest's interactive tutorial.

Course Philosophy

GDD is not a traditional lecture course. You will learn through:

- Interactive in-class workshops
- Challenging take-home machine problems
- Hands-on architecture discussions
- Code review and refactoring sessions

This mirrors professional game development practice.

About GDScript

GDScript is an **interpreted, optionally statically-typed** language tightly integrated with Godot. Key characteristics:

Type System

- Variables are dynamically typed by default (Python-like)
- Type inference occurs at first assignment
- Optional type hints enforce static typing
- Automatic type hint and inference using the walrus operator (`:=`)
- Mix static and dynamic typing as needed

Mandatory Typing Requirements

This course requires type annotations for:

- All function/method signatures
- All class member variables
- Additional variables when clarity demands it

Benefits: Early error detection, better autocomplete, clearer intent in pair programming.

Naming Conventions

GDScript follows Python conventions:

- **Methods and variables:** `snake_case`
- **Classes:** `PascalCase`
- **Private members:** `_private_var` (underscore prefix)
- **Source files:** `lower_snake_case.gd`

Note: GDScript has no true `private` modifier (Python-like). Underscore is convention only, but must be respected.

Example Code

```
const PI_CONST: float = 3.14  # Constant declaration
var radius: float = 2.0       # Typed variable

func get_area_of_circle(r: float) -float:
    # Type inferred from expression and made strict through walrus
    var area := PI_CONST * r ** 2
    return area

# Private method convention
func _calculate_internal() -void:
    pass
```

Common Pitfalls

- Forgetting return type annotations
- Using camelCase instead of snake_case
- Assuming underscore prefix enforces privacy (it does not)
- Inconsistent typing (mixing typed and untyped parameters)

Task

Complete **all 27 sections** of the GDQuest interactive course:

https://gdquest.github.io/learn-gdscript/

Grading Criteria

- **Completeness:** All 27 sections finished
- **Code quality:** Proper naming conventions demonstrated
- **Understanding:** Ability to explain concepts during verification

Grading

Provide evidence of completion to the instructor. Be prepared to discuss key concepts during the workshop.

Preparing for Future Sessions

Required Equipment

- Laptop capable of running Godot 4 (stable release)
- Check system requirements for **compatibility** mode
- Alternative: Use laboratory computers if unavailable

Installation Steps

1. Download Godot 4 from https://godotengine.org/
2. Install and verify it launches correctly
3. Create a test project to confirm functionality

Godot is Lightweight

Godot runs on modest hardware. If your machine can handle basic programming tasks, it likely meets minimum requirements. The compatibility renderer is optimized for older systems.

What to Expect

Future sessions will follow this structure:

- Instructor demonstrates the task live
- Students code alongside, ask questions, suggest improvements
- Focus on scalable design over quick solutions
- Emphasis on code quality and professional practices

This is not a tutorial course. You will be expected to:

- Read documentation independently
- Debug your own code before asking for help
- Explain architectural decisions
- Critique code (yours and others')

Language Flexibility

While GDScript is mandatory for course activities, advanced students may experiment with:

- C# via Godot's .NET version
- C++ through GDExtension
- Python through third-party bindings

This is optional and should only be attempted for portfolio projects after mastering GDScript.

This activity is to be accomplished in class.